

OnSale — эксплуатация

1. Окружения

Окружение	Адрес	Ветка	Контейнер / порт	Каталог на сервере
Боевое	https://onsale.itinrf.ru	main	onsale-app-1 , 127.0.0.1:3000	/opt/onsale
Демо	https://onsaledemo.itinrf.ru	demo	onsale-demo , 127.0.0.1:3001	/opt/onsale-demo
Локально	http://localhost:3000	любая	npm run dev	папка проекта

Репозиторий: GitHub `mrfist95-sketch/RemoteSale` . Ветка `demo` = `main` + демо-режим (баннер, демо-пользователи, `docker-compose.demo.yml` , `prisma/seed-demo.ts`); изменения вносят в `main` и затем вливают в `demo` .

Снаружи приложение доступно только через nginx (TLS, HSTS). Порты контейнеров слушают только `127.0.0.1` . Данные боевой БД живут в docker-томе (`onsale_app-data`), демо — в `onsale-demo_app-demo-data` .

2. Запуск

Локально (разработка)

```
cp .env.example .env          # заполнить NEXTAUTH_SECRET, ADMIN_EMAIL, ADMIN_PASSWORD
npm install
npx prisma generate
npm run db:migrate             # создаёт/обновляет prisma/dev.db
npm run seed                   # администратор + справочник причин
npm run dev                    # http://localhost:3000
```

Windows PowerShell: если `npm.ps1` заблокирован политикой выполнения, используйте `npm.cmd` ... или `cmd` .

Команды: `npm test` (vitest), `npm run typecheck` , `npm run lint` , `npm run build` .

Docker

```
docker compose up -d --build
```

Обязательные переменные в `.env` рядом с compose-файлом: `NEXTAUTH_SECRET` , `ADMIN_PASSWORD` (≥ 8 символов); также `NEXTAUTH_URL` , `ADMIN_EMAIL` . Без них compose не поднимется — защита от «пустого» продакшена.

Что происходит при старте контейнера (`docker-entrypoint.sh`):

1. `node scripts/db-migrate.mjs` — бэкап БД и применение недостающих миграций (§3);

2. `npm run seed` — идемпотентно создаёт администратора из `ADMIN_EMAIL/ADMIN_PASSWORD` и справочник причин (на демо вместо него — `seed-demo.ts`);
3. `scripts/watchdog.mjs` — запускает `next start` и каждые 15 с опрашивает `/api/health`; после 3 неудач подряд перезапускает приложение; при остановке контейнера корректно завершает его (до 10 с). Настройки: `WATCHDOG_INTERVAL`, `WATCHDOG_MAX_FAILS`.

Сборка образа (`Dockerfile`) выполняет `npm ci`, `prisma generate`, `next build` и **прогоняет тесты**: упавший тест = образ не соберётся = деплой остановится до запуска нового кода.

Healthcheck контейнера — `/api/health` (интервал 30 с). Рестарт: `restart: unless-stopped`.

Логи: json-file, 10 МБ × 3.

Пароль администратора из `.env` применяется только при **создании** учётной записи; для уже существующего админа `seed` лишь подтверждает роль — пароль меняйте через интерфейс.

3. Изменение схемы БД

Схема меняется только миграциями, `prisma db push --accept-data-loss` не используется.

Как внести изменение

1. Правьте `prisma/schema.prisma`. Новые поля существующих таблиц — **в конец модели** и с `@default` /необязательные: тогда миграция — простой `ALTER TABLE ... ADD COLUMN`, без пересоздания таблицы (SQLite).
2. Создайте миграцию: `npx prisma migrate dev --name <описание>` (локально). Просмотрите SQL: в нём не должно быть `DROP TABLE` /пересоздания таблиц с данными, если это не задумано.
3. Закоммитьте папку `prisma/migrations/<...>` вместе со схемой.
4. Проверьте на копии боевой БД (§6), затем деплойте.

Что делает `scripts/db-migrate.mjs` при каждом старте

1. Если файл БД существует — копирует его в `<папка БД>/backups/pre-migrate-<время>.db` (хранятся 10 последних).
2. Если БД создана старым способом (есть таблицы, но нет истории миграций) — помечает `baseline 20260828031841_init` как применённую (`migrate resolve --applied`); если колонки `blocked/sessionVersion` и таблица `AuditLog` уже есть, помечает применённой и вторую миграцию.
3. Выполняет `prisma migrate deploy` — применяет только недостающие миграции. Если миграция не прошла, команда падает, контейнер не стартует, а бэкап из п.1 остаётся нетронутым.

4. Деплой

На сервере лежит скрипт `/root/onsale-deploy.sh <каталог> <compose-файл> <контейнер> <порт>`. Он выполняет:

1. **Бэкап БД** из работающего контейнера: `VACUUM INTO` в `/root/backups/...` (согласованная копия без остановки).
2. **Метка отката**: `docker tag <образ> <образ>:rollback-<время>` — предыдущий образ сохраняется.
3. `git fetch` + `git merge --ff-only origin/<ветка>` (деплоится ровно то, что в GitHub; расхождения истории не допускаются).

4. `docker compose build` (с прогоном тестов) → `up -d`.
5. Ожидание статуса `healthy`; при неудаче скрипт сообщает об ошибке (откат — §5).

Порядок релиза:

```
# локально
npm test && npm run typecheck && npm run build
git push origin main
git checkout demo && git merge main && git push origin demo && git checkout main

# на сервере
/root/onsale-deploy.sh /opt/onsale docker-compose.yml onsale-app-1 3000 # б
/root/onsale-deploy.sh /opt/onsale-demo docker-compose.demo.yml onsale-demo 3001 # д
```

Сначала выкатывайте **демо** и проверяйте; боевую — после.

Проверка после деплоя: `curl -s http://127.0.0.1:<порт>/api/health` → `{"ok":true,"db":"ok",...}`; вход под тестовой учёткой; открыть список заказов.

5. Резервное копирование и откат

Бэкапы

- перед каждым деплоем — `/root/backups/` на сервере (`VACUUM INTO`);
- перед каждой миграцией — `backups/pre-migrate-*.db` внутри тома БД (10 последних);
- перед ручным вмешательством делайте копию сами:

```
```bash
```

```
docker exec onsale-app-1 sqlite3 /app/data/prod.db "VACUUM INTO '/app/data/manual-$(date +%F).db'"
```

```
```
```

(если `sqlite3` нет в образе — `docker cp` файла при остановленном контейнере.)

Откат кода (БД не менялась или миграции аддитивные)

```
cd /opt/onsale
docker tag <образ>:rollback-<время> <образ>:latest # вернуть прежний образ
docker compose up -d --no-build
```

Аддитивные миграции (новые колонки/таблицы) старому коду не мешают.

Откат данных — только при порче БД: остановить контейнер, заменить `prod.db` нужной копией (из `/root/backups` или `backups/pre-migrate-*`), запустить. Потеряются изменения, сделанные после снимка. После восстановления — проверка целостности:

```
sqlite3 prod.db "PRAGMA integrity_check; PRAGMA foreign_key_check;" # ожидается ok и пусто
```

6. Проверка на копии боевых данных

Перед выкатыванием миграции:

```
cp <копия prod.db> prisma/check.db
DATABASE_URL="file:./check.db" node -r dotenv/config scripts/db-migrate.mjs
```

Сравните количество записей по таблицам до и после (users, orders, order_items, payments, products, categories, status logs) — они должны совпасть; `integrity_check` = ok, `foreign_key_check` пуст.

7. Тесты

`npm test` — 91 тест (vitest, без внешних сервисов):

| Файл | Что проверяет |
|---|--|
| <code>lib/order-rules.test.ts</code> | таблица переходов статусов по ролям, откат из PAID, отмена |
| <code>lib/tz.test.ts</code> | границы дня и периодов в <code>Asia/Novosibirsk</code> , «день в день» |
| <code>lib/rbac-core.test.ts</code> ,
<code>payments.test.ts</code> | роли, долг/просрочка, допуск округления |
| <code>lib/price-parse.test.ts</code> ,
<code>article.test.ts</code> , <code>password.test.ts</code> ,
<code>rate-limit.test.ts</code> | разбор прайса и кодировок, артикулы, пароли, лимит входа |
| <code>actions.integration.test.ts</code> | настоящие server actions на настоящей SQLite , созданной из миграций; вход подменяется моком <code>getSessionUser</code> . Покрывают: доступ чужих ролей и чужих записей (IDOR), гонки оплат и нумерации заказов, откаты статусов при оплатах, блокировка пользователя, валидация входа |

Интеграционные тесты каждый раз строят БД из `prisma/migrations`, поэтому заодно проверяют, что миграции применяются с нуля.

8. Типовые операции

| Задача | Как |
|-------------------------------|---|
| Сбросить пароль пользователя | Админ → Пользователи → изменить → новый пароль (старые сессии пользователя завершатся) |
| Заблокировать сотрудника | Админ → Пользователи → «Заблокировать»; вход запрещён сразу, сессии сброшены |
| Исправить ошибочную оплату | Продавец — в день внесения; позже — администратор (оплата → корректировка/удаление с причиной) |
| Восстановить отменённый заказ | Только администратор (статус → «Внесён») |
| Загрузить новый прайс | Админ → Прайс-лист → загрузить файл → решить по новым категориям → применить |
| Узнать, кто что менял | журналы заказа (статусы, оплаты, корректировки) и AuditLog (пользователи, оплаты без заказа) |
| Смотреть логи | <code>docker logs --tail 200 onsale-app-1</code> (ошибки действий помечены <code>[action]</code> , watchdog — <code>[watchdog ...]</code>) |
| Приложение «зависло» | watchdog перезапустит само (3 неудачи <code>/api/health</code>); вручную: <code>docker restart onsale-app-1</code> |

9. Ограничения и известные компромиссы

- Один процесс/один контейнер (очередь записи SQLite и лимит входа живут в памяти); перезапуск сбрасывает счётчики попыток входа.
- Деньги — Float (с округлением и допуском); перевод на целые копейки отложен.
- Контейнер запускается от root; одноэтапная сборка Docker (образ крупнее необходимого).
- Нет CI: тесты запускаются при сборке образа и вручную.
- PDF-инструкции в docs/pdf создаются из docs/*.md скриптом scripts/md2pdf.mjs и могут отставать от markdown — пересоздавайте при изменениях руководств.
- Сертификат Let's Encrypt продлевает certbot автоматически; срок текущего — до 2026-12-01.